

The Main.java program never exits when I choose option 6 — it just prints 'Program ended.' and shows the menu again. Walk through the loop logic to find why.

Examined loop termination logic to pinpoint exit condition failure >

Let's trace it exactly as the JVM would execute it.

The loop structure:

```
java
public void runProgram() {
    int choice;

    do {
        showMainMenu();
        choice = inputChoice("Choose option: ", 1, 6);

        switch (choice) {
            case 1: playerList.managePlayers(scanner); break;
            case 2: trainingSessionList.manageTrainingSessions(scanner); break;
            case 3: matchRecordList.manageMatchRecords(scanner); break;
            case 4: contractList.manageContracts(scanner, playerList); break;
            case 5: salaryList.manageSalaries(scanner, playerList); break;
            case 6:
                System.out.println("Program ended.");
                break;
        }
    } while (choice != 7);

    scanner.close();
}
```

The loop structure:

```
java

public void runProgram() {
    int choice;

    do {
        showMainMenu();
        choice = inputChoice("Choose option: ", 1, 6);

        switch (choice) {
            case 1: playerList.managePlayers(scanner); break;
            case 2: trainingSessionList.manageTrainingSessions(scanner); break;
            case 3: matchRecordList.manageMatchRecords(scanner); break;
            case 4: contractList.manageContracts(scanner, playerList); break;
            case 5: salaryList.manageSalaries(scanner, playerList); break;
            case 6:
                System.out.println("Program ended.");
                break;
        }
    } while (choice != 7);

    scanner.close();
}
```

Step-by-step trace when the user enters 6:

1. `inputChoice("Choose option: ", 1, 6)` runs:

```
java
↓
private int inputChoice(String message, int min, int max) {
```

Step-by-step trace when the user enters 6:

1. `inputChoice("Choose option: ", 1, 6)` runs:

```
java
private int inputChoice(String message, int min, int max) {
    while (true) {
        try {
            ...
            int number = Integer.parseInt(scanner.nextLine().trim());
            if (number >= min && number <= max) {
                return number;    // min=1, max=6 → 6 is accepted, returns 6
            }
            ...
        } ...
    }
}
```

With `min = 1, max = 6`, the value `6` satisfies `number >= min && number <= max`, so this method returns `6`. This is the first thing to notice — the method signature itself caps valid input at `6`, it can never produce `7`.

2. Back in `runProgram()`: `choice = 6`.
3. The `switch` matches `case 6:`, prints `"Program ended."`, hits `break;` — which only exits the `switch`, not the `do-while`.
4. Execution falls through to the loop's exit check:

```
java
} while (choice != 7);
```

This evaluates `6 != 7` → `true` → loop continu...

This evaluates `6 != 7` → `true` → loop continues.

5. Control jumps back to the top of the `do` block, calls `showMainMenu()` again, and prompts for input again — even though the user already chose "Exit."

Root cause: The loop's continuation condition (`choice != 7`) and the menu's actual exit value (`6`) don't match. `7` is unreachable — `inputChoice` is hard-bounded to `[1, 6]`, so `choice` can never be `7` under any input. The loop's termination condition is checking for a value that the program is structurally incapable of producing, making `while (choice != 7)` equivalent to `while (true)`.

The fix — make the condition match the actual exit value:

```
java
} while (choice != 6);
```

This is consistent with every other manager class in your codebase (`ContractList`, `SalaryList`, `TrainingSessionList` all correctly use `while (selectedFunction != 6)` / `while (choice != 6)`), which is what makes this bug stand out as an inconsistency — `Main` is the only loop using the wrong sentinel value.

Want me to apply that fix to the file now?